

Reviewer Pack — Minimal Rank of Eichler Orders over Frege Proof Trees

Entry fe912a4106ed · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-26 16:56:05 UTC

Minimal Rank of Eichler Orders over Frege Proof Trees

Entry ID: fe912a4106ed **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-26 16:56:05 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Number Theory (Eichler Orders) **Field B** (complexity object): Complexity Theory: Frege Proof Depth

Statement:

[‘For any n -variables Frege proof tree with depth d , the minimal rank of an Eichler order for its clause indicator polynomial modulo p is at least $cp^{(n-d/2)}$, where c is a constant independent of n .’, ‘Equivalently, if the minimal rank of an Eichler order $r(\text{Frege})$ exists for the clause indicator polynomial of a Frege proof tree F , then $r(\text{Frege}) \geq cp^{(n-d/2)}$.’]

Rationale (proposer’s reasoning):

[‘Eichler orders are algebraic objects that encode arithmetic information about number fields. Their minimal rank could reveal non-trivial arithmetic properties related to the complexity of Frege proofs.’, ‘The conjecture links the arithmetic structure of number fields with the combinatorial complexity of propositional logic, potentially uncovering new insights into proof complexity.’]

Taxonomy category: AlgebraicNumberTheory × ComplexityTheory (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 8bc6b360d0fbb836

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, for all n -variables Frege proof trees with depth d , the mean of the minimal rank of Eichler orders for their clause indicator polynomial modulo p exceeds $cp^{(n-d/2)}$ by at least a statistically significant margin across 30 random seeds. The conjecture is falsified if this condition fails or if any seed produces a minimal rank less than $cp^{(n-d/2)}$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 3 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - title:(Eichler*orders AND Frege*proof*depth) AND (minimal*rank OR rank*) - author:*(Frege*proof*depth AND Eichler*order*polynomial) - subject:Algebraic*Number*Theory AND Frege*proof*tree AND Eichler*order*rank

Top relevant hits considered: - [http://arxiv.org/abs/1105.2538v1] Van der Waals energy surface of carbon nanotubes sheet - [http://arxiv.org/abs/1902.02049v2] On the faces of the tensor cone of symmetrizable Kac-Moody lie algebras - [http://arxiv.org/abs/2006.00298v2] Reconfiguration of District Heating Network for Operational Flexibility Enhancement in Power System Unit Commitment

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=124, elapsed=241.7s

5.1 Generated Python source

```
import random
import math

def gcd(a, b):
    while b:
        a, b = b, a % b
    return a

def lcm(a, b):
    return abs(a * b) // gcd(a, b)

def extended_gcd(a, b):
    if a == 0:
        return b, 0, 1
    else:
        g, x, y = extended_gcd(b % a, a)
        return g, y - (b // a) * x, x

def mod_inverse(a, m):
    g, x, _ = extended_gcd(a, m)
    if g != 1:
        raise ValueError("Modular inverse does not exist")
    else:
        return x % m

def matrix_mod_inv(matrix, p):
    n = len(matrix)
    det = 0
    for i in range(n):
```

```

        det += matrix[0][i] * matrix_minor(matrix, 0, i) * (-1) ** (0 + i)
    det = det % p
    inv_det = mod_inverse(det, p)
    adjugate = [[matrix_minor(matrix, j, i) * (-1) ** (j + i) for i in range(n)] for j in range(n)]
    inverse = [[(adjugate[i][j] * inv_det) % p for j in range(n)] for i in range(n)]
    return inverse

def matrix_minor(matrix, row, col):
    minor = [row[:col] + row[col+1:] for row in matrix[1:]]
    return determinant(minor)

def determinant(matrix):
    n = len(matrix)
    if n == 1:
        return matrix[0][0]
    elif n == 2:
        return matrix[0][0] * matrix[1][1] - matrix[0][1] * matrix[1][0]
    else:
        det = 0
        for i in range(n):
            det += ((-1) ** i) * matrix[0][i] * determinant([row[:i] + row[i+1:] for row in matrix[1:]])
        return det

def gaussian_elimination(matrix, b, p):
    n = len(b)
    augmented_matrix = [[matrix[i][j] for j in range(n)] + [b[i]] for i in range(n)]

    for i in range(n):
        pivot_row = max(range(i, n), key=lambda r: abs(augmented_matrix[r][i]))
        augmented_matrix[i], augmented_matrix[pivot_row] = augmented_matrix[pivot_row], augmented_matrix[i]

        for j in range(i + 1, n):
            factor = (augmented_matrix[j][i] * mod_inverse(augmented_matrix[i][i], p)) % p
            for k in range(n + 1):
                augmented_matrix[j][k] = (augmented_matrix[j][k] - factor * augmented_matrix[i][k]) % p

    x = [0] * n
    for i in range(n - 1, -1, -1):
        x[i] = (augmented_matrix[i][-1] - sum(augmented_matrix[i][j] * x[j] for j in range(i + 1, n))) % p

    return x

def run_trial(seed: int) -> dict:
    random.seed(seed)

    n = random.choice([20, 25, 30, 35, 40])
    d = random.randint(1, n)
    p = 7 # Example prime number
    c = 1.0 # Constant independent of n

    # Generate a random Frege proof tree (simplified for demonstration)
    def generate_frege_tree(depth):
        if depth == 0:
            return random.choice(['x', 'y', 'z'])
        else:
            op = random.choice(['&', '|'])

```

```

        left = generate_frege_tree(depth - 1)
        right = generate_frege_tree(depth - 1)
        return (op, left, right)

def clause_indicator_polynomial(tree):
    if isinstance(tree, str):
        return {tree: 1}
    else:
        op, left, right = tree
        if op == '&':
            return **clause_indicator_polynomial(left), **clause_indicator_polynomial(right)
        elif op == '|':
            left_poly = clause_indicator_polynomial(left)
            right_poly = clause_indicator_polynomial(right)
            result = {}
            for var in set(left_poly.keys()).union(set(right_poly.keys())):
                if var in left_poly and var in right_poly:
                    result[var] = (left_poly[var] * right_poly[var]) % p
                elif var in left_poly:
                    result[var] = left_poly[var]
                else:
                    result[var] = right_poly[var]
            return result

def polynomial_to_matrix(poly):
    n_vars = len(poly)
    matrix = [[0] * n_vars for _ in range(n_vars)]
    for var, coeff in poly.items():
        i = ord(var) - ord('x')
        matrix[i][i] += coeff
    return matrix

tree = generate_frege_tree(d)
poly = clause_indicator_polynomial(tree)
matrix = polynomial_to_matrix(poly)

try:
    rank = len(gaussian_elimination(matrix, [1] * n, p))
except Exception as e:
    return {
        "metric_name": "rank",
        "metric_value": 0,
        "instances_tested": 1,
        "conjecture_holds": False,
        "counterexample": str(e)
    }

expected_rank = math.ceil(c * p ** (n - d / 2))

return {
    "metric_name": "rank",
    "metric_value": rank,
    "instances_tested": 1,
    "conjecture_holds": rank >= expected_rank,
    "counterexample": "" if rank >= expected_rank else f"rank={rank}, expected={expected_rank}"
}

```

```

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std=0.0 support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results) and support_fraction >= 0.8:
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"first failing seed\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE reason=insufficient_evidence")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out before producing data, which means it did not complete its intended analysis to verify the conjecture. | next: Re-run the test with increased time limits or optimize the code to ensure it completes within a reasonable timeframe.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	11251
2	preregistration	ollama_remote	glm4:latest	0	0	6213
3	novelty	ollama_remote	glm4:latest	0	0	4929
4	novelty	ollama_remote	glm4:latest	0	0	5863

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	19272
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18175
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	28073
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	19996
9	judge	ollama_remote	glm4:latest	0	0	63658

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 177430 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/fe912a4106ed.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/fe912a4106ed.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/fe912a4106ed.tar.gz (if generated)